Adding styles to React app

class vs className

- **JSX is NOT real HTML**. *JSX is just fancy JavaScript disguised to look like HTML*. When Vite builds your project, it *converts all your JSX* into pure, **raw JavaScript functions**.
- **The Problem with class in JavaScript** :
    - In JavaScript, **class is a reserved keyword** used for Object-Oriented Programming (like creating ES6 classes):

```
class User {
    constructor(name) {
        this.name = name;
        }
}
```

    - Because class is already a strictly reserved word in JavaScript, the creators of React could not use class as an attribute in JSX without causing a massive conflict in the JavaScript engine!
    - React converts className="note" back into real HTML:
    - !

Improved error message

- Learnt Here how to replace ugly browser alerts into Beautiful notification with logic.

```
const Notification = ({ message }) => {
  if (message === null) {
    return null
  }

  return (
    <div className="error">
      {message}
    </div>
  )
}
export default Notification
```

- In React, **if a component returns null**, **React completely removes it from the browser DOM**. Zero HTML is drawn!

Inline Styles

- CSS rules are defined slightly differently in JavaScript than in normal CSS files
- traditional Way (**regular CSS, is that hyphenated (kebab case)**)

```
{
  color: green;
  font-style: italic;
}
```

- React Way (object) - (**CSS properties are written in camelCase.**)

```
{
  color: 'green',
  fontStyle: 'italic'
}
```

E.g Styles Component in react way

```
const Footer = () => {
  const footerStyle = {
    color: 'green',
    fontStyle: 'italic'
  }

  return (
    <div style={footerStyle}>
      <br />
      <p>
        Note app, Department of Computer Science, University of Helsinki 2025
      </p>
    </div>
  )
}

export default Footer
```

- Inline styles comes with certain limitations.

    1. **No Pseudo-classes (:hover, :active, :focus):** You cannot write hover effects with inline styles! In CSS, you write button:hover { background: red }. In an inline style object, :hover does not exist.

    2.** No Media Queries (Responsive Mobile Design)🤫* You cannot write @media (max-width: 768px) in an inline style object to change styles on mobile phones vs desktop screens.

3. **No Keyframe Animations:** You cannot define CSS @keyframes animations directly inside a style={{}} object.
4. **Code Clutter & Performance:** *If a component needs 20 styles*, your JSX becomes massive and unreadable, and *JavaScript has to create a brand new style object in memory on every single render*.
5. **Takeaway:** Use inline styles only for tiny, dynamic things (like changing a progress bar width style={{ width:${percent}% }}). For real design, use external CSS files, CSS Modules, or Tailwind.

Index.css vd App.css

- index.css (Global Styles for the entire website)
  - Where it is imported: Inside main.jsx (import './index.css').
  - What goes inside it: Global, baseline styles that apply to the entire webpage, such as:
    - Setting the font family for the whole website: body { font-family: sans-serif; }
    - CSS resets / box-sizing: * { box-sizing: border-box; margin: 0; }
    - Global CSS variables (:root { ... })
    - Global utility classes (like your .error or .success notification classes)
- App.css (Styles specifically for the App Component)
  - Where it is imported: Inside App.jsx (import './App.css').
  - What goes inside it: Styles that specifically belong to the layout of , such as:
    - Centering the main app container: .app-container { max-width: 600px; margin: auto; }
    - Vite's demo animations / logos
